

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Algorithm Design Challenge

COURSE ARTIFICIAL INTELLIGENCE COURSE CODE: MLA0101
NAME: AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO1: Apply search algorithms and heuristic techniques to solve state-space search and optimization problems. (BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100

Time: 90 Mins

No. of Questions: 5

Annexure A Algorithm Design Challenge

Code of Conduct:

I Yoga Madha A-192425058 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Criterion	Weightage	Marks Obtained
Problem Analysis and Understanding	15%	
AI Algorithm Selection & Justification	15%	
Algorithm Design / Pseudocode	25%	
Application of AI Concepts (Greedy, A*, CSP, Minimax, etc.)	15%	
Diagram / State Space / Search Tree Representation	10%	
Complexity Analysis and Solution Efficiency	10%	
Originality, Critical Thinking, and Presentation	10%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

Answer all the Questions

1.Design an algorithm using Greedy Best-First Search to find a path from a source node to a goal node for the given graph. Clearly define the heuristic function, show the node expansion order, and justify why Greedy Best-First Search is suitable for this problem.

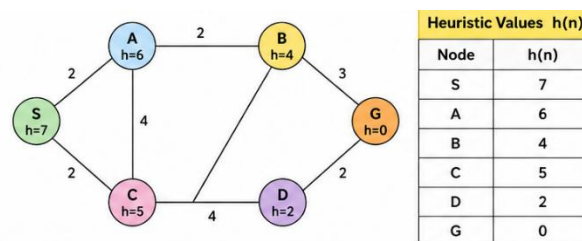
Answer:

1. Problem Statement

The objective is to design an **Informed Search algorithm using Greedy Best-First Search (GBFS)** to find a path from a given **source node** to a **goal node** in a graph.

Greedy Best-First Search uses a **heuristic function $h(n)$** to estimate how close each node is to the goal. At every step, it selects the node with the **smallest heuristic value** and expands it. Therefore, the algorithm tries to reach the goal as quickly as possible by always choosing the node that appears closest to the goal.

For illustration, consider the following graph:



2. Problem Analysis

Initial State

The search begins at node S.

Goal State

The desired goal is node G.

Assumptions

- The graph contains connected nodes.
- Each node represents a possible state.
- Edges represent possible movements between states.
- A heuristic value is available for each node.
- The goal node has heuristic value 0.

Constraints

The algorithm should:

- Start from the source node.
- Select the most promising node based on its heuristic.
- Avoid repeatedly expanding already visited nodes.
- Continue until the goal is reached or no nodes remain.

3. AI Technique Selection

The selected technique is **Greedy Best-First Search (GBFS)**.

Greedy Best-First Search is an **informed search algorithm** because it uses additional problem knowledge in the form of a heuristic function.

Its evaluation function is:

$$f(n)=h(n)$$

where:

- $f(n)$ = priority of node (n)
- $h(n)$ = estimated cost from node (n) to the goal

The node having the **smallest $h(n)$** is selected for expansion.

4. Justification of Algorithm

Greedy Best-First Search is suitable because the problem requires finding a path from a source to a goal while using information about how close each node is to the goal.

Unlike **Breadth-First Search**, which explores nodes level by level, GBFS uses heuristic information to move toward the goal.

Unlike **Depth-First Search**, GBFS does not simply follow one branch deeply. Instead, it selects the currently most promising node.

Compared with A*, GBFS is simpler because it considers only the estimated remaining cost:

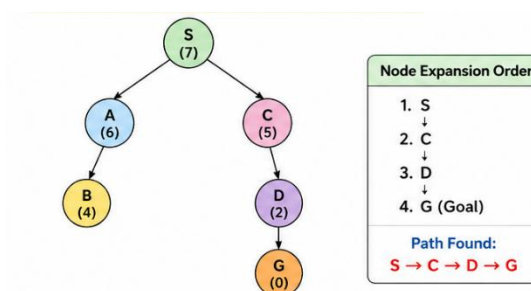
$$f(n)=h(n)$$

whereas A* considers both the cost already travelled and the estimated remaining cost:

$$f(n)=g(n)+h(n)$$

Therefore, GBFS can often reach the goal quickly, although it does **not guarantee the shortest path**.

5. State Space Representation



6. Variables, Domains, and Constraints

Not Applicable.

This is a graph-search problem and not a **Constraint Satisfaction Problem (CSP)**.

7. Heuristic Function

The heuristic function estimates the remaining distance from the current node to the goal.

$$h(n)=\text{estimated cost from node } n \text{ to goal}$$

For this example:

Node Heuristic $h(n)$

S	7
A	6
B	4
C	5
D	2
G	0

The goal has:

$$h(G)=0$$

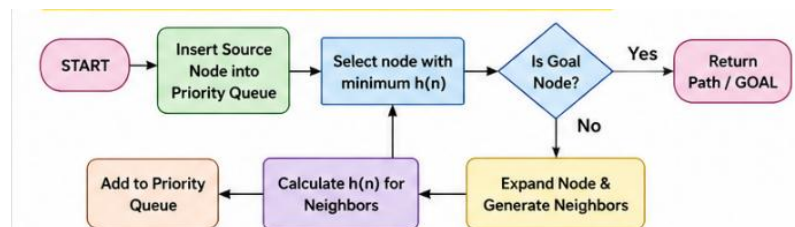
because there is no remaining distance once the goal is reached.

8. Algorithm Design / Pseudocode

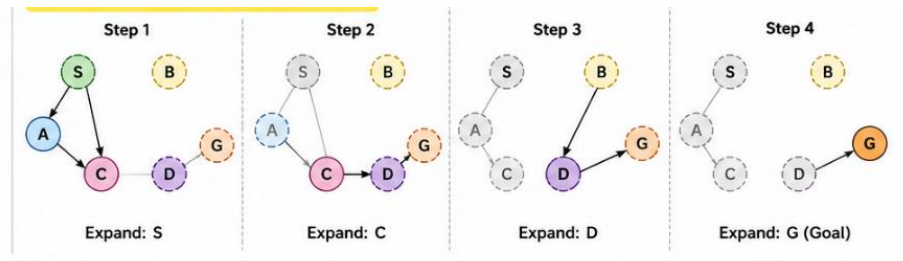
Greedy Best-First Search Algorithm

1. Start with the source node.
2. Insert the source node into the priority queue.
3. Assign priority based on $h(n)$.
4. While the priority queue is not empty:
 - a. Remove the node with the smallest $h(n)$.
 - b. If the node is the goal:
return the path.
 - c. Mark the node as visited.
 - d. Generate all unvisited neighboring nodes.
 - e. Calculate $h(n)$ for each neighbor.
 - f. Insert the neighbors into the priority queue.
5. If the queue becomes empty:
return "No path found".

9. Flowchart / Algorithm Diagram

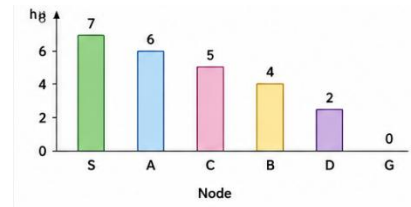


10. Working / Execution Trace



Execution Table

Step	Expanded Node	$h(n)$	Frontier (Priority Queue)	Selected Next
1	S	7	C(5), A(6)	C
2	C	5	D(2), A(6)	D
3	D	2	G(0), A(6)	G
4	G	0	–	Goal Reached



11. Complexity Analysis

For Greedy Best-First Search, let:

(b) = branching factor

(m) = maximum depth of the search tree

Time Complexity

In the worst case:

$$O(b^m)$$

The algorithm may need to explore a large number of nodes if the heuristic leads it in an unhelpful direction.

Space Complexity

Greedy Best-First Search stores frontier nodes and visited nodes.

Therefore, the worst-case space complexity is approximately:

$$O(b^m)$$

It can require considerable memory for large search spaces.

12. Advantages and Limitations

Advantages

1. **Uses heuristic information** to guide the search toward the goal.
2. Usually reaches the goal faster than uninformed searches when the heuristic is good.
3. Simple evaluation function:

$$f(n)=h(n)$$

4. Suitable for large search spaces where exploring every node is inefficient.
5. Easy to implement using a priority queue.

Limitations

1. It does **not guarantee the shortest path**.
2. Its performance depends heavily on the quality of the heuristic.
3. A poor heuristic can lead the search in the wrong direction.
4. It can consume significant memory.
5. It may explore unnecessary nodes when several nodes have similar heuristic values.

13. Conclusion

Greedy Best-First Search is an effective **informed search technique** for finding a path from a source node to a goal node when a suitable heuristic function is available. In this example, the algorithm uses ($h(n)$) to select the node that appears closest to the goal. Starting from **S**, the node expansion order is **S** $\rightarrow$ **C** $\rightarrow$ **D** $\rightarrow$ **G**, successfully reaching the goal.

The main advantage of GBFS is its ability to focus the search toward the goal instead of blindly exploring the entire graph. However, because it considers only ($h(n)$) and ignores the cost already travelled, it does not guarantee an optimal path. **A*** can be used as an alternative when both path cost and heuristic information need to be considered.

Heuristic Values of Search Nodes

Lower heuristic values are preferred by Greedy Best-First Search.

node	Heuristic
S	7
A	6
C	5
B	4
D	2
G	0

2.A delivery robot must travel from a warehouse to a customer through a weighted road network. Design an A Search algorithm to determine the optimal path. Calculate $g(n)$, $h(n)$, and $f(n)$ for each expanded node, and explain why A guarantees the optimal solution.

Answer:

1. Problem Statement

A delivery robot must travel from a **warehouse** to a **customer** through a weighted road network. Each road has a different travel cost or distance. The objective is to find the **optimal path**, meaning the path with the minimum total cost from the warehouse to the customer.

A* Search is used because it combines the cost already travelled with an estimated cost remaining to the goal.

For **A***, the evaluation function is:

$$f(n)=g(n)+h(n)$$

where:

- ($g(n)$) = actual cost from the warehouse to node (n)
- ($h(n)$) = estimated cost from node (n) to the customer
- ($f(n)$) = estimated total cost of the solution through node (n)

2. Problem Analysis

Initial State

The robot starts at the **Warehouse (W)**.

Goal State

The robot must reach the **Customer (G)**.

Assumptions

- Each road has a non-negative cost.
- The road network can be represented as a weighted graph.
- The heuristic gives an estimate of the remaining cost.
- The heuristic is admissible, meaning it does not overestimate the actual remaining cost.

Constraints

- The robot must start from W.
- The robot must reach G.
- The total travelling cost should be minimized.
- Already explored nodes should not be unnecessarily expanded.

3. AI Technique Selection

The selected technique is **A* Search**.

A* is an **informed search algorithm** that uses both:

$$g(n) \text{ and } h(n)$$

to decide which node should be expanded next.

Its evaluation function is:

$$f(n)=g(n)+h(n)$$

The node with the **smallest $f(n)$** is selected for expansion.

4. Justification of Algorithm

A* Search is suitable because the delivery robot needs to find an **optimal path** in a weighted graph.

Greedy Best-First Search uses only:

$$f(n)=h(n)$$

and may choose a path that looks close to the goal but has a high travelling cost.

A* considers both:

cost already travelled + estimated remaining cost

Therefore:

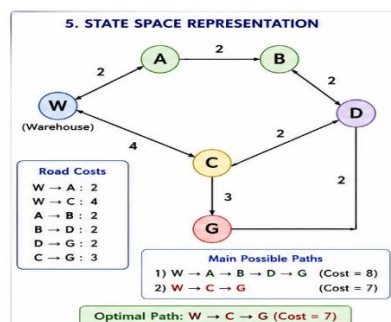
$$f(n)=g(n)+h(n)$$

This allows A* to balance the distance already travelled with the estimated distance to the customer.

When the heuristic is **admissible** and the path costs are non-negative, A* can guarantee an optimal solution.

5. State Space Representation

Consider the following weighted road network:



6. Variables, Domains, and Constraints

Not Applicable.

This is a weighted graph-search problem and not a Constraint Satisfaction Problem.

7. Heuristic Function

Node	$h(n)$ (Estimated cost to G)
W	6
A	5
B	3
C	2
D	2
G	0

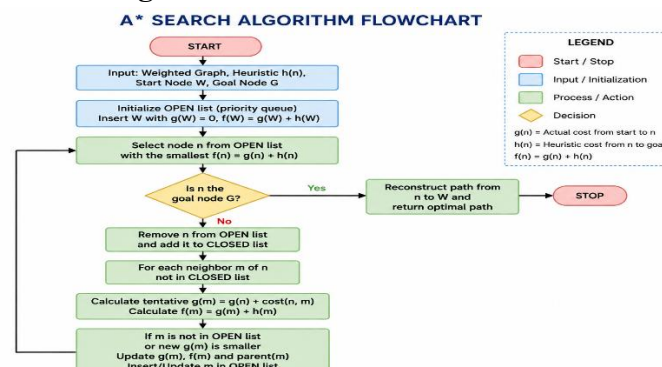
Admissible Check:
For every node n , $h(n) \leq$ actual minimum cost to G
(Hence, heuristic is admissible.)

8. Algorithm Design / Pseudocode

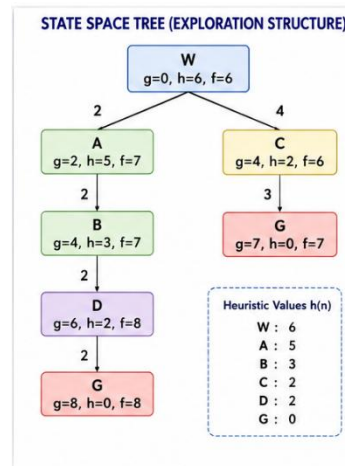
A* Search Algorithm

1. Start from the warehouse W.
2. Set $g(W) = 0$.
3. Calculate $f(W) = g(W) + h(W)$.
4. Insert W into the priority queue.
5. While the priority queue is not empty:
 - a. Select the node with the smallest $f(n)$.
 - b. If the selected node is the goal:
return the optimal path.
 - c. Expand the selected node.
 - d. For each neighboring node:
Calculate the new $g(n)$.
Calculate $h(n)$.
Calculate $f(n) = g(n) + h(n)$.
 - e. Add/update the neighbor in the priority queue.
6. If the queue becomes empty:
return "No path found".

9. Flowchart / Algorithm Diagram



10. Working / Execution Trace



Complete Execution Table

Step	Expanded Node	$g(n)$	$h(n)$	$f(n)$	Next Choice
1	W	0	6	6	C
2	C	4	2	6	G
3	G	7	0	7	Goal

11. Complexity Analysis

Let:

- (b) = branching factor
- (d) = depth of the optimal solution

Time Complexity

The worst-case time complexity of A* is approximately:

$$O(b^d)$$

The actual performance depends strongly on the quality of the heuristic.

A good heuristic reduces the number of nodes that need to be expanded.

Space Complexity

A* stores the frontier and explored nodes, so its worst-case space complexity is:

$$O(b^d)$$

Therefore, A* can require significant memory for very large road networks.

12. Advantages and Limitations

Advantages

1. **Finds an optimal path** when the heuristic is admissible and the relevant cost conditions are satisfied.
2. Uses both actual and estimated costs.
3. Usually performs better than uninformed search.
4. A good heuristic can significantly reduce unnecessary node expansion.
5. Very suitable for **robot navigation, GPS routing, game pathfinding, and delivery planning**.

Limitations

1. It can require a large amount of memory.
2. A poor heuristic can make the algorithm slower.
3. Designing a good heuristic can be difficult.
4. For very large graphs, computation and memory requirements can become high.
5. The algorithm is more complex than simple BFS or DFS.

13. Conclusion

A* Search is an appropriate algorithm for finding the optimal route of a delivery robot in a weighted road network. It evaluates each node using:

$$f(n)=g(n)+h(n)$$

where $(g(n))$ represents the actual cost travelled and $(h(n))$ estimates the remaining cost to the customer. A* guarantees an optimal solution when the heuristic is **admissible** (and, for graph search, appropriately consistent or handled with node re-opening). This makes A* particularly useful for delivery robots because it considers both the distance already travelled and the estimated distance remaining, rather than relying only on the apparent closeness to the destination.

3.A university needs to prepare an examination timetable without scheduling conflicting subjects at the same time. Design the problem as a Constraint Satisfaction Problem (CSP) by identifying variables, domains, and constraints. Explain how MRV and Forward Checking improve the efficiency of the solution.

Answer:

1. Problem Statement

A university needs to prepare an examination timetable for several subjects. The main requirement is that **two subjects having common students must not be scheduled at the same time**.

The timetable can be modeled as a **Constraint Satisfaction Problem (CSP)**. Each subject is treated as a variable, possible examination time slots form its domain, and conflicts between subjects are represented as constraints.

The objective is to assign a valid time slot to every subject while satisfying all examination constraints.

2. Problem Analysis

Initial State

Initially, no examination has been assigned a time slot.

For example:

Subject A → ?

Subject B → ?

Subject C → ?

Subject D → ?

Subject E → ?

Goal State

The goal is a **complete timetable** in which every subject has a time slot and no conflicting subjects occur simultaneously.

Assumptions

- There are a fixed number of examination slots.
- Each subject must be assigned exactly one slot.
- Subjects with common students are considered conflicting.
- A student cannot attend two conflicting examinations at the same time.
- Each slot can accommodate the required examinations, unless a capacity constraint is specified.

Constraints

If two subjects have common students, they cannot have the same time slot.

3. AI Technique Selection

The appropriate AI technique is a **Constraint Satisfaction Problem (CSP)**.

A CSP consists of:

Variables + Domains + Constraints

For this examination problem:

- **Variables** → Subjects
- **Domains** → Available examination slots
- **Constraints** → Conflicting subjects must have different slots

Two important CSP techniques used are:

Minimum Remaining Values (MRV)

MRV selects the variable having the **smallest remaining legal domain**.

Forward Checking

After assigning a value to a variable, Forward Checking removes that value from the domains of neighboring conflicting variables.

These techniques reduce unnecessary search.

4. Justification of Algorithm

A CSP is suitable because the examination timetable is not simply a path-finding problem. Instead, several variables must receive values while satisfying a set of restrictions.

A basic **Backtracking Search** can solve the problem, but it may try many unnecessary assignments.

MRV improves this process by selecting the most constrained subject first.

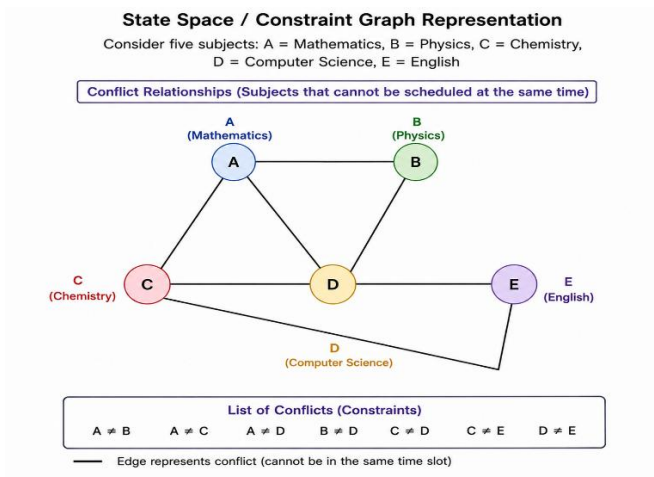
Forward Checking detects conflicts immediately after an assignment.

Therefore:

CSP + MRV + Forward Checking

provides a more efficient solution than ordinary trial-and-error assignment.

5. State Space / Constraint Graph Representation



6. Variables, Domains, and Constraints

This is the most important part of the CSP formulation.

Variables

Each subject is a variable:

$$X = \{A, B, C, D, E\}$$

where:

- A = Mathematics
- B = Physics
- C = Chemistry
- D = Computer Science
- E = English

Domains

Each subject initially has three possible examination slots:

$$D(A) = D(B) = D(C) = D(D) = D(E) = \{T1, T2, T3\}$$

Therefore:

Variable	Subject	Initial Domain
A	Mathematics	{T1, T2, T3}
B	Physics	{T1, T2, T3}
C	Chemistry	{T1, T2, T3}
D	Computer Science	{T1, T2, T3}
E	English	{T1, T2, T3}

7. Heuristic Function

For a CSP, the main heuristic used here is **MRV** rather than a traditional numerical heuristic ($h(n)$).

Minimum Remaining Values

MRV calculates:

$$MRV(X) = |D(X)|$$

where ($D(X)$) is the current legal domain of variable X.

The variable with the **smallest domain** is selected first.

For example:

$A \rightarrow \{T1, T2\} \rightarrow 2 \text{ values}$
 $B \rightarrow \{T1, T2, T3\} \rightarrow 3 \text{ values}$
 $C \rightarrow \{T1, T3\} \rightarrow 2 \text{ values}$
 $D \rightarrow \{T1\} \rightarrow 1 \text{ value}$
 $E \rightarrow \{T2, T3\} \rightarrow 2 \text{ values}$

Why MRV is useful

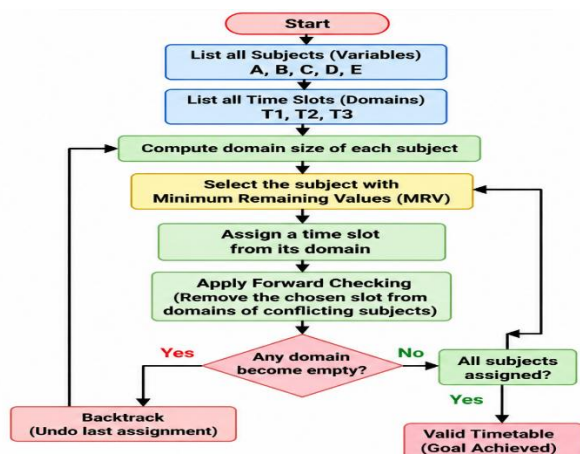
A variable with fewer choices is more likely to cause a failure. Selecting it first allows the algorithm to discover impossible assignments early instead of wasting time assigning less-constrained variables.

8. Algorithm Design / Pseudocode

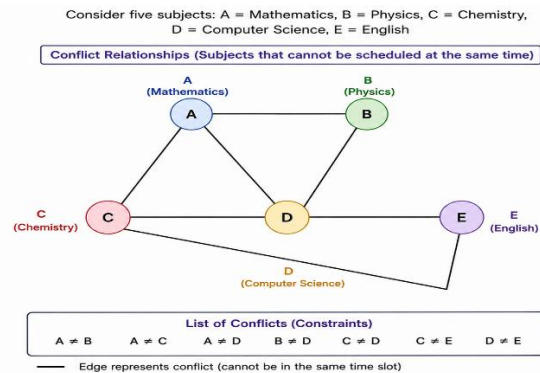
CSP with MRV and Forward Checking

1. Start with all subjects unassigned.
2. While there are unassigned subjects:
 - a. Select the unassigned subject with the smallest remaining domain using MRV.
 - b. Select a possible time slot.
 - c. Assign the time slot to the subject.
 - d. Apply Forward Checking:
 - Remove the selected slot from the domains of all conflicting unassigned subjects.
 - e. If any domain becomes empty:
 - Reject the assignment.
 - Backtrack to the previous assignment.
 - f. Otherwise continue.
3. If all subjects are assigned:
 - Return the timetable.
4. If no valid assignment exists:
 - Report failure.

9. Flowchart / Algorithm Diagram



10. Working / Execution Trace



11. Complexity Analysis

For a CSP with:

- (n) variables
- (d) possible values per variable

the worst-case search space is:

$$O(d^n)$$

because every variable may potentially take every domain value.

Time Complexity

Worst-case backtracking complexity:

$$O(d^n)$$

MRV and Forward Checking do not change the theoretical worst-case exponential nature, but they can **significantly reduce the practical number of assignments explored**.

Space Complexity

The search can require approximately:

$$O(n+d)$$

space for assignments, domains, and search information, depending on the implementation.

12. Advantages and Limitations

Advantages of CSP

- Naturally represents timetable problems.
- Clearly separates variables, domains and constraints.
- Can handle many subjects and examination slots.
- Finds assignments satisfying all specified constraints.

Advantages of MRV

- Selects the most constrained variable first.
- Detects failures earlier.
- Reduces unnecessary search.
- Helps reduce the branching factor.

Advantages of Forward Checking

- Removes impossible values immediately.
- Prevents obviously invalid future assignments.
- Detects domain failure early.

- Reduces unnecessary backtracking.

Limitations

- CSP search can still be exponential for large problems.
- A large number of subjects can produce a very large search space.
- Additional constraints such as room capacity, faculty availability, student preferences and holidays make the problem more complex.
- MRV may require repeatedly checking domain sizes.
- Forward Checking adds extra computation for maintaining domains.

13. Conclusion

The university examination timetable can be effectively modeled as a **Constraint Satisfaction Problem**. The **subjects are variables**, the available examination periods are their **domains**, and conflicts between subjects form the **constraints**.

The basic CSP can be solved using backtracking, but **MRV** improves the process by selecting the subject with the fewest available time slots first. **Forward Checking** then removes the selected slot from the domains of conflicting subjects. If any subject has no remaining valid slot, the algorithm immediately backtracks.

Thus:

CSP + MRV + Forward Checking

provides an efficient approach for generating a conflict-free examination timetable while reducing unnecessary search.

4.Design the Minimax algorithm for the given game tree to determine the best move for the MAX player. Then apply Alpha–Beta Pruning to the same tree and indicate which nodes are pruned. Compare the number of nodes evaluated before and after pruning.

Answer:

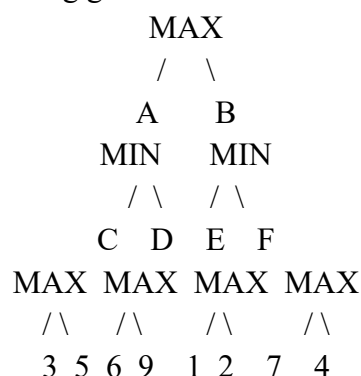
1. Problem Statement

The problem is to determine the **best move for the MAX player** in a given game tree. MAX tries to **maximize the final score**, while MIN tries to **minimize the score**.

First, the **Minimax algorithm** is applied to evaluate the complete game tree and determine the best move for MAX.

Then, **Alpha–Beta Pruning** is applied to the same tree. Alpha–Beta Pruning avoids evaluating branches that cannot affect the final decision. The number of nodes evaluated before and after pruning is then compared.

For illustration, consider the following game tree:



The leaf values represent the utility/payoff values of the game.

2. Problem Analysis

Initial State

The initial state is the **root node**, controlled by MAX.

Goal State

The goal is to select the move that gives MAX the **highest guaranteed utility**.

MAX chooses the maximum value:

$$\text{MAX}(a,b)=\max(a,b)$$

MIN chooses the minimum value:

$$\text{MIN}(a,b)=\min(a,b)$$

Assumptions

- MAX and MIN play optimally.
- The game tree is finite.
- Leaf nodes contain utility values.
- MAX tries to maximize the score.
- MIN tries to minimize the score.

3. AI Technique Selection

The selected techniques are:

1. **Minimax**
2. **Alpha–Beta Pruning**

Minimax

Minimax is used in two-player, zero-sum games where one player tries to maximize the score and the other tries to minimize it.

Alpha–Beta Pruning

Alpha–Beta Pruning improves Minimax by eliminating branches that cannot influence the final decision.

The two important values are:

- **Alpha ((alpha))** → best value currently guaranteed for MAX.
- **Beta ((beta))** → best value currently guaranteed for MIN.

4. Justification of Algorithm

Minimax is appropriate because the game involves two competing players with opposite objectives.

MAX wants:

Maximum utility

while MIN wants:

Minimum utility

Therefore, the algorithm alternates between MAX and MIN levels.

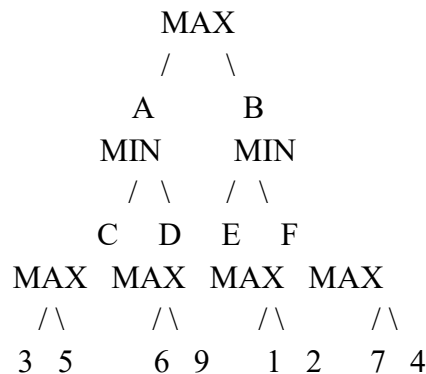
However, standard Minimax evaluates every leaf node. This can be expensive for large game trees.

Alpha–Beta Pruning improves Minimax because it eliminates branches that cannot possibly change the final decision.

Importantly, Alpha–Beta Pruning produces **the same optimal move as Minimax**, but usually evaluates fewer nodes.

5. State Space / Game Tree Representation

Consider:



The tree contains:

- 1 root MAX node
- 2 MIN nodes
- 4 MAX nodes
- 8 leaf nodes

Total nodes:

$$1+2+4+8= 15$$

6. Variables, Domains, and Constraints

Not Applicable.

This is a **game-playing problem**, not a Constraint Satisfaction Problem.

7. Heuristic Function

Not Applicable.

The given tree already contains utility values at the leaf nodes, so a heuristic estimate is not required.

The utility values are:

3, 5, 6, 9, 1, 2, 7, 4

8. Algorithm Design / Pseudocode

Minimax Pseudocode

MINIMAX(node, maximizingPlayer)

if node is a terminal node:

return utility(node)

if maximizingPlayer:

best = $-\infty$

for each child of node:

value = MINIMAX(child, False)

best = max(best, value)

```

    return best
else:
    best =  $+\infty$ 
    for each child of node:
        value = MINIMAX(child, True)
        best = min(best, value)
    return best

```

Alpha-Beta Pseudocode

ALPHABETA(node, α , β , maximizingPlayer)

if node is a terminal node:

 return utility(node)

if maximizingPlayer:

 value = $-\infty$

 for each child:

 value = max(value,

 ALPHABETA(child, α , β , False))

α = max(α , value)

 if $\alpha \geq \beta$:

 break // prune remaining children

 return value

else:

 value = $+\infty$

 for each child:

 value = min(value,

 ALPHABETA(child, α , β , True))

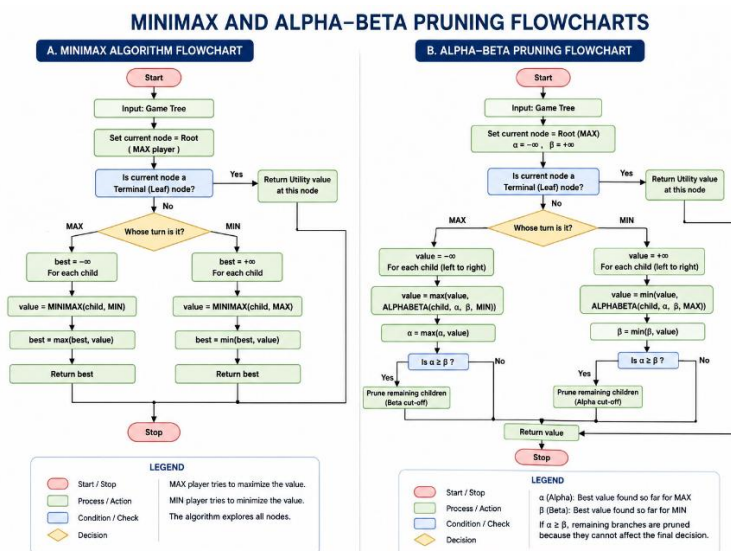
β = min(β , value)

 if $\alpha \geq \beta$:

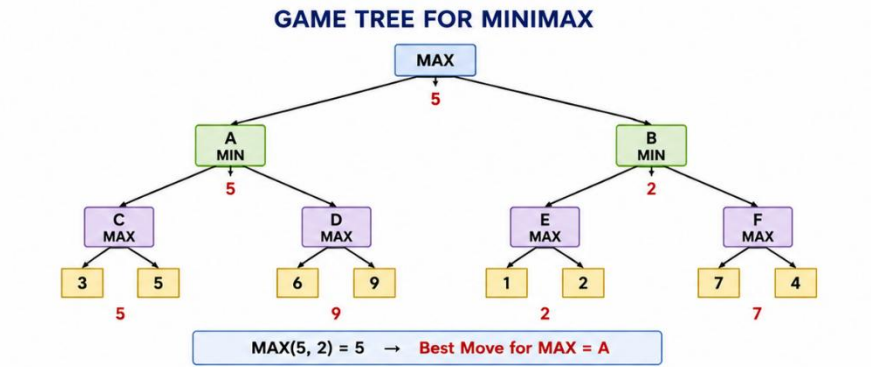
 break // prune remaining children

 return value

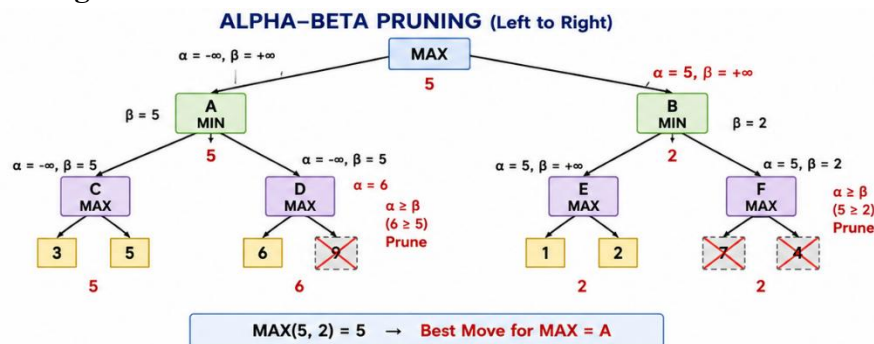
9. Flowchart / Algorithm Diagram



10. Working / Execution Trace



Alpha-Beta Pruning



Comparison Table

Method	Nodes Evaluated	Nodes Pruned
Minimax	15	0
Alpha-Beta Pruning	12	3

11. Complexity Analysis

Let:

- (b) = branching factor
- (d) = depth of the game tree

Minimax

Time complexity:

$$O(b^d)$$

Space complexity for a depth-first implementation:

$$O(b^d)$$

Alpha-Beta Pruning

In the worst case, if poor move ordering prevents effective pruning:

$$O(b^d)$$

In the best case, with excellent move ordering:

$$O(b^{\{d/2\}})$$

Thus, Alpha-Beta Pruning can potentially search **much deeper** than ordinary Minimax within the same computational resources.

12. Advantages and Limitations

Minimax Advantages

- Simple and systematic.
- Suitable for two-player adversarial games.
- Guarantees the optimal move when the complete game tree is searched.
- Considers the opponent's best possible response.

Minimax Limitations

- Evaluates many unnecessary nodes.
- Becomes expensive for large game trees.
- Requires significant computation as depth increases.

Alpha–Beta Advantages

- Produces the **same result as Minimax**.
- Reduces the number of nodes evaluated.
- Allows deeper game-tree searches.
- Does not change the final optimal decision.
- Particularly effective with good move ordering.

Alpha–Beta Limitations

- Its effectiveness depends on the ordering of moves.
- Worst-case complexity is still $O(b^d)$.
- Implementation is slightly more complex than basic Minimax.

13. Conclusion

When **Alpha–Beta Pruning** is applied, the same optimal decision is obtained, but several unnecessary leaf nodes are not evaluated. In the given left-to-right traversal, **3 nodes are pruned**, reducing the evaluated nodes from **15 to 12**, a **20% reduction**.

Thus, Alpha–Beta Pruning makes Minimax more efficient without changing the quality of the final decision.

5.A warehouse robot operates in a dynamic environment where obstacles may appear unexpectedly. Design an intelligent search strategy using either Local Search or an Online Search Agent. Explain how the algorithm adapts to environmental changes and compare its performance with informed search algorithms.

Answer:

1. Problem Statement

A warehouse robot must move through a warehouse to collect and deliver packages. However, the environment is **dynamic**, meaning obstacles such as other robots, workers, boxes, or temporary barriers may appear unexpectedly.

A traditional search algorithm plans a path using the information available at the beginning. If an obstacle suddenly appears, the planned path may no longer be valid.

For this problem, an **Online Search Agent** is suitable because the robot can **observe the environment while moving**, update its knowledge, and choose a new path when conditions change.

The objective is to reach the destination safely while minimizing unnecessary movement and adapting to newly appearing obstacles.

2. Problem Analysis

Initial State

The robot starts at its current warehouse position.

For example: $S=(1,1)$

Goal State

The robot must reach the package/customer location: $G=(6,6)$

Environment

The warehouse contains:

- Storage shelves
- Walking paths
- Other robots
- Workers
- Temporary obstacles
- Package locations

Some obstacles are known initially, while others may appear unexpectedly.

Assumptions

- The robot has sensors such as cameras, LiDAR, or proximity sensors.
- The robot can detect nearby obstacles.
- The robot can move in different directions.
- The robot can update its map while operating.
- The destination remains reachable.

Constraints

- The robot must avoid collisions.
- It should reach the destination.
- It should minimize travel distance/time.
- It must react when an unexpected obstacle blocks its current route.

3. AI Technique Selection

The selected technique is an **Online Search Agent**.

Why Online Search?

The warehouse environment is dynamic, so a fixed path calculated beforehand may become invalid.

An online search agent continuously updates its knowledge and adapts its actions according to the current environment.

4. Justification of Algorithm

A conventional informed search algorithm such as A^* can find an efficient path when the environment is sufficiently known.

However, suppose A^* calculates:

$S \rightarrow A \rightarrow B \rightarrow C \rightarrow G$

and a worker places a box at B.

The original path becomes:

$S \rightarrow A \rightarrow [\text{Obstacle}] \rightarrow C \rightarrow G$

A static plan cannot simply continue through B.

An online search agent detects the obstacle and searches for an alternative:

$S \rightarrow A \rightarrow D \rightarrow E \rightarrow C \rightarrow G$

Therefore, online search is more appropriate for environments where information changes during execution.

5. State Space Representation

A simple warehouse grid can be represented as:

	1	2	3	4	5	6
1	S					
2			X			
3			X			
4					X	
5					X	
6						G

S = Starting position

G = Goal

X = Obstacle

Initially, the robot may know the fixed obstacles.

While moving, it can discover new obstacles.

For example:

Before movement:

S → → → → → G

Unexpected obstacle appears:

S → → X → → G

Robot:

S → ↓ → → ↑ → G

The robot changes its route after observing the obstacle.

6. Variables, Domains, and Constraints

Not Applicable.

This is an **online search and dynamic path-planning problem**, not a CSP.

7. Heuristic Function

Although online search does not require a fixed complete map, a heuristic can still be used to select promising movements.

A common heuristic for a grid environment is **Manhattan distance**:

$$h(n) = |x_n - x_G| + |y_n - y_G|$$

where:

- (x_n, y_n) = current position
- (x_G, y_G) = goal position

This gives an estimate of the remaining distance.

However, unlike a traditional offline A* algorithm, the online agent **recalculates its decision when new environmental information becomes available**.

8. Algorithm Design / Pseudocode

Online Search Agent

1. Start at the initial position S.
2. Observe the surrounding environment.
3. Identify available movements.
4. Estimate the cost of each possible movement.
5. Select the most promising safe movement.
6. Move one step.
7. Observe the environment again.
8. If a new obstacle is detected:

Update the environment model.
 Mark the blocked path as unavailable.
 Recalculate an alternative route.

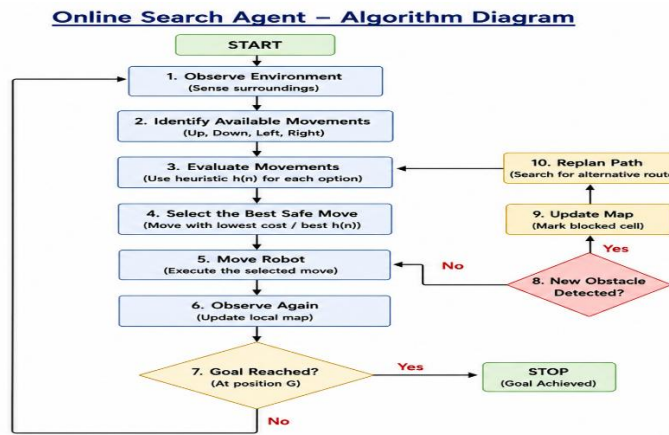
9. If the goal is reached:

Stop.

10. Otherwise:

Repeat from Step 2.

9. Flowchart / Algorithm Diagram



10. Working / Execution Trace

Consider the following initial route:

$S \rightarrow A \rightarrow B \rightarrow C \rightarrow D \rightarrow G$

The robot begins at S.

Step 1: Initial Observation

The robot observes:

$S \rightarrow A \rightarrow B \rightarrow C \rightarrow D \rightarrow G$

No obstacle blocks the planned route.

Step 2: New Obstacle Appears

While moving toward B, the robot's sensor detects a new obstacle:

$S \rightarrow A \rightarrow X \rightarrow C \rightarrow D \rightarrow G$

where X represents the unexpected obstacle.

The robot does not continue toward B.

Instead, it updates its map:

B = BLOCKED

Step 3: Replanning

The robot searches for an alternative route.

Suppose an alternative route is:

$A \rightarrow E \rightarrow F \rightarrow C \rightarrow D \rightarrow G$

Step 5: Another Obstacle

Suppose another robot suddenly blocks F.

The agent observes:

$A \rightarrow E \rightarrow X \rightarrow C$

It updates its map again and searches for another safe movement.

11. Complexity Analysis

For an online search agent, complexity depends on:

- Number of states explored
- Number of available actions
- Frequency of environmental changes
- Quality of the heuristic
- Amount of previously stored information

If (b) is the branching factor and (d) is the search depth, a general search process may have worst-case complexity around: $O(b^d)$

The exact complexity depends on the specific online search algorithm used.

The memory requirement is approximately related to the number of discovered states: $O(b^d)$ in the worst case.

However, practical online implementations can restrict the search to a **local region** around the robot, reducing computation.

12. Advantages and Limitations

Advantages

1. Adapts to Dynamic Environments

The robot can react when new obstacles appear.

2. Uses Current Sensor Information

The robot does not depend entirely on an outdated map.

3. Supports Real-Time Decision Making

The robot can make decisions while moving.

4. Handles Unknown Areas

The robot can discover previously unknown parts of the warehouse.

5. Replanning

If the current route becomes blocked, the agent can select another route.

Limitations

1. May Produce a Longer Path

Repeated replanning can result in a longer route than an ideal path calculated with complete information.

2. Computational Cost

Frequent replanning requires additional computation.

3. Sensor Dependence

Incorrect or incomplete sensor information can result in poor decisions.

4. Dynamic Obstacles

If obstacles move frequently, the robot may need to replan continuously.

5. No Guaranteed Globally Optimal Path

Because the complete environment is not known in advance, the robot may not immediately find the globally shortest possible route.

13. Conclusion

An **Online Search Agent** provides an intelligent solution for a warehouse robot operating in a dynamic environment. Unlike a fixed search strategy, the robot continuously **observes, moves, detects changes, updates its map, and replans** when necessary.

When an unexpected obstacle appears, the robot marks the blocked path as unavailable and searches for a safe alternative instead of continuing with an invalid plan.

EVALUATION RUBRICS

Criteria	Weightage (%)	Excellent (90–100%)	Good (75–89%)	Average (60–74%)	Poor (<60%)
Problem Analysis and Understanding	15%	13–15% – Clearly identifies the problem, initial state, goal state, constraints, and expected outcome.	10–12% – Identifies most problem components with minor omissions.	7–9% – Demonstrates partial understanding with some missing elements.	0–6% – Incorrect or incomplete understanding of the problem.
AI Algorithm Selection & Justification	15%	13–15% – Selects the most appropriate AI algorithm/search technique with strong technical justification.	10–12% – Selects an appropriate algorithm with adequate justification.	7–9% – Selects a partially suitable algorithm with limited justification.	0–6% – Incorrect algorithm selection or no justification.
Algorithm Design / Pseudocode	25%	22–25% – Designs a complete, logical, efficient, and error-free algorithm/pseudocode.	17–21% – Algorithm is mostly correct with minor logical errors.	12–16% – Algorithm is partially complete with several logical errors.	0–11% – Algorithm is incomplete, incorrect, or difficult to understand.
Application of AI Concepts (Greedy, A*, CSP, Minimax, Alpha-Beta, etc.)	15%	13–15% – Correctly applies the relevant AI concepts and techniques throughout the solution.	10–12% – Applies AI concepts correctly with minor mistakes.	7–9% – Demonstrates limited application of AI concepts.	0–6% – Unable to apply the appropriate AI concepts or techniques.
Diagram / State Space / Search Tree Representation	10%	9–10% – Accurate, complete, and well-labelled diagram supporting the solution.	7–8% – Diagram is mostly correct with minor omissions.	5–6% – Diagram is partially complete or lacks clarity.	0–4% – Diagram is missing or incorrect.
Complexity Analysis and Solution Efficiency	10%	9–10% – Correctly analyzes time and space complexity and proposes an efficient solution.	7–8% – Complexity analysis is mostly correct with minor errors.	5–6% – Partial complexity analysis or limited discussion of efficiency.	0–4% – No meaningful complexity analysis or highly inefficient solution.

Originality, Critical Thinking, and Presentation	10%	9–10% – Demonstrates excellent originality, logical reasoning, organized presentation, and explores optimized/alternative solutions.	7–8% – Demonstrates good critical thinking with a clear presentation.	5–6% – Shows limited originality and acceptable presentation.	0–4% – Poor presentation with little or no critical thinking.
--	-----	--	---	---	---